

SME Portal — Metrics Decoder

What this is: one line per number on every SME screen — what it literally counts, and what it does *not*. Written so you can answer "what is this number?" in a meeting without opening code.

Audience: us + the SME portal team. **Source of truth:** `src/modules/sme/services/*` in this repo. Anything the portal computes itself is marked **[portal-side]**.

0. Read this first — why two screens can show different "premium" or "active" numbers

There are **three separate data planes**. Nothing reconciles them, and that is by design.

PLANE	WHERE IT COMES FROM	COVERS	RETENTION
A — Domain tables ("engaged")	6 real product tables: <code>simulation_attempts</code> , <code>user_content</code> , <code>user_question_attempts</code> , <code>custom_tasks</code> (only where <code>source_id IS NULL</code>), <code>psychometric_test_results</code> , <code>user_document_progress</code>	Only <i>genuine actions</i> . A user who opened the app and scrolled is invisible.	Full history
B — Request capture ("active")	<code>api_usage</code> (raw) + <code>api_usage_daily</code> (rollup)	Every authenticated API call. Public routes, <code>x-api-key</code> routes and webhooks are skipped.	raw = 30 days , rollup = long
C — Billing/user tables	<code>user_auth</code> , <code>orders</code> , <code>payments</code> , <code>payment_events</code>	Money + account state	Full history

Rules of thumb:

- **engaged < active** almost always. Engaged is "did something", active is "made any request".
- Anything sourced from plane B **cannot** go back more than ~30 days. A low number in an old window is *retention*, not a quiet week.
- All day buckets are **IST (UTC+5:30)**. A "day" is midnight-to-midnight India time, not UTC.
- Chat, mains evaluation and PYQ-variation have **zero persistence** — they never appear in any usage number, anywhere.

1. Analytics screen (`/analytics`)

Powered by `GET /sme/analytics` — but this endpoint is **built by the portal**, not by us. It pulls the full `/sme/users` , `/sme/orders` and `/sme/transactions` lists and adds them up in Python. **[portal-side]**

KPI cards

CARD	WHAT IT IS	WATCH OUT
Total Users	Count of every row in <code>user_auth</code> .	⚠ Portal fetches at most 6,000 users (100/ page × 60 pages). Past 6,000 this card silently stops growing — and so does every other number on this screen.

CARD	WHAT IT IS	WATCH OUT
Premium Access	Users with premium access <i>right now</i> = <code>SUBSCRIBED</code> and not expired, OR <code>ACTIVE</code> and still inside the 14-day trial.	Includes trials. This is "who can use paid features", not "who paid".
Paying Subscribers	The <code>premiumState = 'Premium'</code> bucket = <code>SUBSCRIBED</code> and not expired.	The honest revenue-side number. Always ≤ Premium Access.
Revenue (captured)	Sum of <code>CAPTURED</code> payment amounts, in ₹, excluding <code>MANUAL</code> grants.	Manual comps are shown separately in "Revenue by source" so the headline isn't inflated by free grants. ⚠️ This sums the payment ledger, not paid orders — see the note below.
<code>x% onboarded</code>	Share of users with <code>user_profiles.onboarding_completed = true</code> .	Current state, not a cohort.
<code>x% of users</code> (on Premium Access)	Premium Access ÷ Total Users.	
<code>x% success</code> (on Revenue)	Captured payments ÷ all payment rows.	Denominator includes <code>CREATED</code> / <code>FAILED</code> attempts, so a user retrying three times drags it down.

Paid orders vs payments — the check that caught a real bug (2026-07-29). "Revenue (captured)" sums the **payments** table; "Order status → PAID" counts **orders**. They should agree. On 29 Jul they did not: 16 PAID orders against 14 payments. Two Razorpay purchases had been collected with no payment row, so ₹6,499 sat outside the revenue figure — and a refund on them could not have revoked premium, because `handleRefund` resolves the user through payment → order → userId. Root cause was ours, not the portal's: payment rows were only ever created by the client-called `/payments/verify`, both webhook handlers were update-only, and `verifyPayment` short-circuits on `premiumGrantedAt` — so a webhook-first purchase (routine on UPI intent, where the bank confirms before the user returns to the app) could never get a row. **Fixed** in `payments.service.ts` — both webhook handlers now call the idempotent `upsertCapturedPayment` — and **backfilled** **2026-08-06** via `scripts/backfill-missing-razorpay-payments.sql`. The recurring-subscription path was never affected. Keep comparing these two numbers: it is the cheapest revenue-integrity check on the screen.

Alert strips

ALERT	WHAT IT IS	WATCH OUT
N paid order(s) need reconcile	<code>PAID</code> orders where <code>premiumGrantedAt</code> is NULL.	⚠️ The column only shipped 2026-06-16. Every order paid before that date is NULL and counts here forever. Only orders newer than that date are a real alert.
N premium expiring within 7 days	Currently-premium users whose <code>premiumExpiresAt</code> falls in the next 7 days.	Trial users have no <code>premiumExpiresAt</code> , so they never appear.

Bar lists

CHART	EACH BAR IS	WATCH OUT
Subscription funnel	Count of users per <code>premiumState</code> : <code>Premium</code> (paid, live) · <code>Trial</code> (in the 14-day window) · <code>Trial Ended</code> (lapsed, never paid) · <code>Churned</code> (paid once, now expired) · <code>Downloaded</code> (signed up, trial still open but nothing else).	Derived from timestamps, not from the <code>status</code> column — so it's correct even for users who haven't opened the app since expiry. Exactly one bucket per user.

CHART	EACH BAR IS	WATCH OUT
User status	The raw <code>user_auth.status</code> enum (ACTIVE / SUBSCRIBED / SUSPENDED / ...).	This is the <i>stored</i> column and can be stale — <code>status</code> is only refreshed on an authenticated request. Prefer Subscription funnel.
Sign-in provider	Google / Apple / phone, from <code>user_auth.provider</code> .	
Order status	Orders by <code>CREATED</code> / <code>PAID</code> / <code>FAILED</code> .	An abandoned checkout leaves a <code>CREATED</code> row forever, so <code>CREATED</code> is usually the biggest bar and means nothing.
Plan mix	Orders by <code>planType</code> (MONTHLY / ANNUAL).	Counts orders, not people. An Apple annual renewal creates a new order each year.
Payment method	Captured payments by Razorpay method (<code>upi</code> , <code>card</code> , ...) or <code>apple_iap</code> .	
Revenue by source	Captured ₹ split by the parent order's <code>paymentSource</code> : APPLE / RAZORPAY / MANUAL.	A payment whose order wasn't in the fetched page set lands in "Unknown" — a big Unknown bar means the 6,000-row cap bit, not a data problem.

Apple In-App Purchases block

NUMBER	WHAT IT IS	WATCH OUT
Apple revenue	₹ on <code>PAID</code> orders with <code>paymentSource = APPLE</code> .	Includes renewals; each renewal is its own order.
Subscribers	Distinct <code>userId</code> across those paid Apple orders.	Lifetime distinct, not "currently subscribed".
Paid orders (incl. renewals)	Count of those orders.	
Events recorded	Rows in the <code>payment_events</code> ledger.	⚠️ Currently wrong — this counts Razorpay events too. The portal asks for <code>?source=APPLE</code> but its HTTP client overwrites the query string with <code>page / limit</code> , so the filter is dropped. See §8.
New / Renewals / Billing issues / Expired / Refunds	Apple notification types: <code>SUBSCRIBED</code> + <code>OFFER_REDEEMED</code> / <code>DID_RENEW</code> + <code>RENEWAL_EXTENDED</code> / <code>DID_FAIL_TO_RENEW</code> / <code>EXPIRED</code> + <code>GRACE_PERIOD_EXPIRED</code> / <code>REFUND</code> + <code>REVOKE</code> .	These five <i>are</i> correct — they match on an <code>apple.</code> prefix, so the leaked Razorpay rows don't affect them. Only the "Events recorded" total is polluted.

Month bars

CHART	EACH BAR IS	WATCH OUT
New users by month	Signups per calendar month, last 12 months.	Built from the fetched user rows — subject to the 6,000 cap.
Revenue by month (paid orders)	₹ of <code>PAID</code> orders by the month the order was created .	Order-created, not payment-captured. A renewal lands in its own month.

2. Engagement screen (/engagement)

Powered by `GET /sme/analytics/<metric>` — these **are** computed by our backend.

Pulse

NUMBER	WHAT IT IS	WATCH OUT
DAU (24h)	Distinct users with ≥ 1 <i>genuine action</i> (plane A) in the last 24 hours.	Rolling 24h, not "today".
<code>active N</code> (under DAU)	Distinct users who made any authenticated request in 24h (plane B).	Always $\geq$ DAU. If it's null/0, request capture isn't reporting.
WAU (7d) / MAU (30d)	Same as DAU but 7 / 30 days.	
<code>active N</code> (under MAU)	Distinct users in <code>api_usage_daily</code> over 30 days.	
Stickiness	$\text{engaged DAU}(24h) \div \text{engaged MAU}(30d)$.	The standard "how many monthlies show up daily" ratio. 10–20% is normal for a study app.

Daily active users chart

Two lines per IST day: **engaged** (plane A, full history) and **active** (plane B). The active line is **null before request capture shipped** — a gap on the left is missing capture, not zero traffic.

Feature pull

NUMBER	WHAT IT IS	WATCH OUT
actions	Row count per feature in the window: <code>simulation</code> , <code>mcq</code> , <code>doc_reading</code> , <code>psychometric</code> , <code>ai_<type></code> , <code>planner</code> .	<code>planner</code> counts only user-created tasks (<code>source_id IS NULL</code>), never auto-seeded ones.
users	Distinct users per feature.	Chat / mains-eval / PYQ-variation are absent by design — nothing is stored for them.

Monetization × engagement

NUMBER	WHAT IT IS	WATCH OUT
premium / free rows	Users split on <code>premium_expires_at > now</code> .	⚠️ A different definition of "premium" than the Analytics screen. Trial users land in <code>free</code> here (no expiry date), and a churned-but-not-yet-expired user lands in <code>premium</code> . Expect this count to differ from both Analytics cards.
activeUsers	Of those, how many took ≥ 1 action in the window.	
avg actions / active user	$\text{totalActions} \div \text{activeUsers}$.	Denominator is <i>active</i> , not all — so it never gets diluted by dormant accounts.
Paywall hits (by route)	HTTP 402 responses per route, i.e. someone hit a paid wall.	Upgrade intent, not failures. From raw <code>api_usage</code> → max ~30 days regardless of the window you pick.

Retention & risk

NUMBER	WHAT IT IS	WATCH OUT
Cohort size	Users who signed up inside the window.	
D1 / D7 / D30	Of that cohort, how many were active at any point ≥ 1 / ≥ 7 / ≥ 30 days after their own signup .	⚠ Cumulative, not "on day 7" . And someone who signed up 3 days ago <i>cannot</i> satisfy D7 — so a 30-day window always shows a depressed D7/D30. Compare cohorts of equal age, or widen the window.
Churn risk	Users whose last genuine action was more than <code>inactiveDays</code> ago but within the last 90 days .	⚠ Users who never did anything are excluded entirely, and anyone silent >90 days drops off the list. This is "going quiet", not "already gone".

Behaviour

NUMBER	WHAT IT IS	WATCH OUT
Heatmap	Count of requests (not users) per IST weekday $\times$ hour.	Plane B $\rightarrow$ last ~30 days only, whatever window you set.
Top documents / readers	Progress rows per document title in the window.	Grouped by title+subject, so two docs with the same title merge.
Mock completion	Simulation attempts submitted $\div$ started in the window.	An attempt started on day 29 and submitted on day 31 counts as started-not-submitted.
Notification read rate	<code>notification_history</code> rows with <code>isRead = true</code> $\div$ rows sent, in the window.	⚠ <code>isRead</code> means "this user opened the in-app feed at some point after", not "tapped this push". See §5. Topic broadcasts aren't in this table at all.
Endpoints (calls / users)	From the <code>api_usage_daily</code> rollout: total calls and distinct users per route, top 100.	
Platforms	Distinct users with an active device token per platform.	A user with both iOS and web is counted twice. This is device registrations, not installs or sessions.

Video · reels

Straight from the **Mux Data API**, not our database.

NUMBER	WHAT IT IS	WATCH OUT
Total views / Unique viewers	Mux's own counts over the timeframe.	
Watch time	Total watch time including paused and stalled time.	The "actually playing" sub-line is the honest one.
Avg watch / view	Watch time $\div$ views.	
Top videos	Mux's <code>video_id</code> is our <code>playbackId</code> ; titles are joined in from the reels list.	A reel with no title just shows its playback id.

User explorer

COLUMN	WHAT IT IS	WATCH OUT
Actions 30d	Rows in the 6 domain tables (plane A) for that user in the last 30 days.	⚠️ The current UI tooltip says "screen opens, taps & other tracked events" — that is wrong . We do not track screen opens or taps at all. It is only the 6 tables. Worth fixing in the portal.
Last seen	<code>max(sessions.lastAccessedAt)</code> — last time a session token was used.	Closer to "last opened the app" than Actions 30d.
Premium until	<code>premium_expires_at</code> .	NULL for trial users and for never-paid users alike.

3. Insights → Question quality (</insights/question-quality>)

Finds **broken questions**, not hard ones. `GET /sme/content/question-quality`. Default window **90 days**, minimum **20 attempts**.

NUMBER	WHAT IT IS	WATCH OUT
Attempts	Content pool: distinct users who answered (one upserted row per user per question). Simulation pool: answered rows in submitted attempts.	Blank answers are counted as <code>skipped</code> and excluded from attempts.
Wrong	Content pool: the user's latest stored <code>isCorrect</code> . Simulation pool: <code>selectedOption</code> compared against the question's current key.	Someone who missed it, retried and got it right counts as correct — so wrong-rate is a <i>floor</i> . If a key was fixed later, old simulation answers are re-scored under the new key.
Wrong rate %	wrong ÷ attempts.	Raw, no confidence — do not rank on this.
Guessing rate / chance %	<code>1 - 1/optionCount</code> — 75% for a 4-option MCQ.	The bar a well-formed question can't fall below.
Confident wrong-rate %	Wilson 95% lower bound on the wrong rate.	Collapses toward 0 at low sample. "We are 95% sure it's at least this bad."
Flag: <code>worse_than_chance</code>	Confident wrong-rate is above the guessing rate.	A correct question cannot beat random guessing. This is a defect, near-certainly.
Flag: <code>suspicious</code>	Confident wrong-rate above 60% but still below the guessing rate.	Might just be hard. Second priority.
Priority score	<code>max(0, confidentWrongRate - chance) × attempts</code> .	≈ "learners we're 95% sure this item misled <i>beyond</i> what guessing would have cost". A 90%-wrong item seen by 500 outranks a 100%-wrong item seen by 12. 0 on almost everything at low sample — that's correct, not a bug.
Contradicts key	The most-picked option got more votes than the marked answer.	The single strongest "the answer key is wrong" signal. Check these first.
Learners misled (summary)	Sum of priority scores across the filtered set.	A blast-radius total, not a user count.
Eligible	Questions that cleared the minimum-attempts bar and were scored.	The denominator for Worse-than-chance / Suspicious.

4. Insights → Release health (/insights/release-health)

Answers "should we force-update?". `GET /sme/analytics/release-health`. Default window **14 days, hard max 30** (that's the raw-data retention limit).

NUMBER	WHAT IT IS	WATCH OUT
App version	The <code>x-app-version</code> header on each request.	⚠ pre-1.7 (no x-app-version header) is a real cohort, not missing data — the header shipped with 1.7, so every older build lands there. Never delete or merge that row.
Active users	Distinct users who made a request on that build × platform.	A user on two builds in the window counts in both rows.
Requests	Captured authenticated requests for that build.	Public/login routes are not captured — a build that can't get past login is invisible here . Its signal shows up in Client errors.
Error rate (excl. paywall)	$(4xx + 5xx - 402) \div (\text{requests} - 402)$.	This is the number to judge a build on.
Error rate (incl. paywall)	$4xx + 5xx \div \text{requests}$, 402 included.	402 is the paywall doing its job — a monetisation signal, not a fault. Ignore this column when comparing builds.
Client errors	<code>auth_events</code> rows of type <code>client_error</code> reported by the app itself.	Can create a row for a build that made zero successful requests. That's deliberate — a crash-only build is exactly what we're hunting.
Client errors / active user	$\text{clientErrors} \div \text{activeUsers}$.	The crash-proxy.
Adoption share %	This build's active users ÷ all active users on that platform.	⚠ Can sum past 100% across builds, because a user upgrading mid-window counts on both.
vs. previous build	<code>better</code> / <code>worse</code> / <code>similar</code> / <code>insufficient_data</code> against the next-older build on the same platform.	Thresholds: ±1.0 error-rate points or ±0.5 client errors per user . A verdict needs ≥200 requests and ≥5 active users on both builds — otherwise <code>insufficient_data</code> , which means "too quiet to judge", not "fine".
Not derivable (headline)	Shown when there isn't a comparable pair.	

5. Insights → Notification effectiveness (/insights/notification-fx)

`GET /sme/analytics/notification-effectiveness`. Default window **30 days**, maturation **3 days**, kill threshold **≤25%**, min matured sent **50**.

The one caveat that must travel with every number here: `isRead` = "this recipient opened the in-app notification feed at some point after this row was created." It is set **in bulk** for all of a user's unread rows when they open the feed. It does **not** mean they tapped this push, saw this push, or that the push was even delivered — **push delivery is not tracked at all**. Read it as "did this person come back to the app", not "was this notification compelling".

NUMBER	WHAT IT IS	WATCH OUT
Sent	Personal notification rows created in the window.	⚠ Broadcasts are not here . Topic sends live in <code>topic_notifications</code> , which has no read flag — they cannot be measured.

NUMBER	WHAT IT IS	WATCH OUT
Read	Of those, how many have <code>isRead = true</code> .	See the caveat above.
Matured sent / Matured read rate	Only rows older than the maturation window (default 3 days) .	A push sent 20 minutes ago hasn't had a fair chance. All rates are quoted on the matured sample only — this is why recent days look empty.
Pending	Rows too young to judge. Excluded from every rate.	If Matured read rate says "maturing", everything in the window is still pending.
Wasted sends	maturedSent – maturedRead.	Blast radius. The kill-list is ranked by this, not by rate — killing a bad type that goes to 5 people saves nothing.
Recipients	Distinct users who got that type.	Built from the newest 20,000 rows only; the banner says so when the cap is hit.
Types	Distinct <code>type</code> values in the window.	
Worst performers	Types with ≥ 50 matured sends and $\leq 25\%$ read rate.	Tune both thresholds in the toolbar. Empty $\neq$ healthy — check "Matured sent" first.
Time to read	Always "not derivable" .	<code>read_at</code> exists but records the <i>bulk feed open</i> , not this notification's open. Any number here would be fabricated, so we return none.

How to read a low rate honestly: a type sent mostly to already-engaged users scores well regardless of its content. A **very low** rate is the trustworthy direction of this signal; a high rate is weak evidence.

6. Insights → Onboarding funnel (`/insights/onboarding-funnel`)

`GET /sme/analytics/onboarding-funnel` . A **signup cohort**: everyone who signed up in the window, followed to their current state.

STAGE	WHAT "REACHED" MEANS
1. Signed up	<code>user_auth.createdAt</code> in the window. This is the denominator.
2. Phone verified	<code>phone_verified = true</code> right now.
3. Onboarding completed	<code>user_profiles.onboarding_completed = true</code> right now.
4. Psychometric resolved	Completed OR explicitly skipped OR a result row exists. It's "past the gate", not "took the test" — the <code>completed / skipped only / with result row</code> mini-tags break it down.
5. First genuine action	First row in any of the 6 domain tables at or after signup.

NUMBER	WHAT IT IS	WATCH OUT
% of signups	reached $\div$ cohort signups.	
Conversion from previous	reached $\div$ previous stage's reached.	

NUMBER	WHAT IT IS	WATCH OUT
Dropped / drop-off %	<code>previous.reached</code> – <code>reached</code> .	⚠️ Can be negative. Some users satisfy a stage without the one before it (e.g. finished onboarding without a verified phone). We report it raw rather than clamping, and <code>reachedWithoutPrevious</code> tells you how many.
Median / p90 time to reach	Nearest-rank percentile, so it's always a duration someone really experienced.	Stage 3 has no timing at all — there is no <code>onboarding_completed_at</code> column, and inventing one from <code>updated_at</code> would be a lie. Stage 4 times only the <i>completed</i> path; a skip has no timestamp.
Sample coverage %	Share of people at that stage who carry a usable timestamp.	Low coverage = the median describes a minority.
First action <code>bySource</code>	Which of the 6 tables produced the first action.	⚠️ <code>psychometric_test_results</code> is in that union — so a user whose only action was the psychometric test satisfies stage 5 with the same row that satisfied stage 4 . This breakdown is how you see how many.
Auth events (login failed / otp send failed / otp verify failed)	App-wide pre-auth failures for that IST day, from <code>auth_events</code> .	⚠️ Capture began 2026-07-23 — days before that show <code>not rec.</code> , never 0. And coverage is split : <code>otp_send_failed</code> is emitted server-side and is real ; <code>login_attempt</code> / <code>login_success</code> / <code>login_failed</code> / <code>otp_verify_failed</code> are mobile-only and stay at 0 until that app build ships . A zero there means "not recorded", never "no failures".
Login success rate	<code>login_success</code> ÷ <code>login_attempt</code> for the covered days.	Meaningless until the mobile build lands (see above).
Worst-day correlation	The covered day with the worst phone-verification rate, with that day's OTP failures beside it, plus the median OTP-verify failures for comparison.	Needs a day with ≥5 signups . Its whole purpose: put an OTP spike next to the verification drop it caused.

7. Other screens (quick reference)

SCREEN	NUMBER	WHAT IT IS
App Users	<code>premiumState</code> badge	Same 5-bucket funnel label as the Analytics chart.
	<code>trialDaysLeft</code>	Days to the effective trial end (<code>trialEndsAt</code> if an SME extended it, otherwise signup + 14 days).
	Platform filter	Filters on active device tokens , not on sign-in provider. [portal-side] The portal's local filter compares against <code>provider</code> / <code>subscriptionSource</code> instead — different meaning, different results.
Transactions	Amounts	Always ₹. We divide the stored paise by 100 before sending.
	<code>premiumGrantedAt</code> on an order	When premium was actually granted. NULL on every order older than 2026-06-16.
Feedback	Chat thumbs summary	Only rows the app explicitly submitted; the AI conversation itself is not stored.
User detail → Timeline	Merged record across 17 sources	Reverse-chronological, cursor-paginated. Absence of an event means "not captured", not "didn't happen".

8. Known-wrong / misleading numbers (the short list)

Status re-verified live against portal build v2.32.0 on 2026-07-29. That release closed most of the labelling items the same morning. Verify against the live portal, never against a downloaded copy of the portal repo — the snapshot in `~/Downloads` carries no git history and goes stale within days.

Still open

1. **Analytics → Apple "Events recorded" is inflated with Razorpay events.** The portal requests `/sme/webhooks?source=APPLE`, but its HTTP client replaces the whole query string with `page / limit`, dropping the filter. **Proven:** the webhook ledger holds 30 events in total, its first row is `eventType: order.paid, source: razorpay`, and the Apple panel reports exactly 30. **[portal-side]** — pass `source` as a param rather than baking it into the path. The five lifecycle counters below it match on an `apple.` prefix and are unaffected. ⚠️ Do not "prove" this by arithmetic: $3 + 11 + 3 = 17$ against a total of 30 does **not** demonstrate a leak, because other legitimate `apple.*` types would explain the same gap. The proof is a razorpay row inside the ledger being counted.
2. **Analytics is capped at 6,000 rows per list.** Total Users, revenue totals, signups-by-month and every bar on that screen quietly stop growing past that; failed pages are silently dropped too. Not biting yet — 3,989 users at ~68/day — but it starts truncating around **late August 2026**. **[portal-side]** — raise the page cap or move the aggregation to our backend.
3. **Plane-B cards still over-promise their window.** Release Health is now capped to 7/14/30 and the heatmap carries a note, but **paywall hits still offers a 7/30/60/90 selector over ~30 days of data**.
4. **The Onboarding Funnel's stage-5 breakdown renders `[object Object]` and `NaN%`** — the portal is stringifying an array of objects. Still present on v2.32.0. The 1,079 headline above it is sound.

Closed

1. **"Paid orders need reconcile" is historically inflated** — `premiumGrantedAt` only exists from 2026-06-16. Addressed by labelling: the Orders tab now explains that an order predating the field can look unreconciled even though premium was granted.
2. **Three different "premium" numbers across two screens** — the Engagement screen now states outright that its tier means "premium expiry > now, so active trials count as free — a different rule from Analytics' two premium figures".
3. **"Actions 30d" claims screen opens and taps** — replaced with the actual list of counted features, plus an explicit "screen opens and taps are not tracked".
4. **D7/D30 retention looks broken on a short window** — the screen now says so itself: "Anyone who signed up fewer than N days ago cannot qualify yet but still counts in the cohort ... Not a broken metric."
5. **Auth-event zeros in the funnel** — capture is live and reporting (140 attempts / 117 successes on 29 Jul); pre-capture days render as "not rec." rather than 0.
6. **Two Razorpay payments missing from the ledger** — ours, not the portal's. Fixed in `payments.service.ts` and backfilled 2026-08-06. See the note under §1.